Argument-Driven Engineering: Using Claims and Evidence to Direct AI-Assisted Work

Emmett Miller

***Abstract*—AI assistants can propose engineering changes and the tests intended to assess them. A team must still determine whether those changes satisfy a useful requirement and whether the available evidence justifies the next action. This article proposes argument-driven engineering as a way to organise that work. A versioned engineering case links an intended outcome to scoped claims, the evidence they require and the reasoning that connects them. Unresolved obligations direct implementation or investigation. Obtained observations support a decision only within the declared conditions. The proposal builds on evidence-based software engineering and assurance cases while retaining the activities of the software development lifecycle. EngArg and EAL-MCP provide existing foundations for argument construction, evidence acquisition and durable assessment. A general work scheduler and systematic reassessment after system changes remain extensions. A hypothetical retry-safe submission feature illustrates the method. A comparative study template specifies how to assess decision quality and total engineering cost. The proposed benefit remains an empirical hypothesis.**

***Index Terms*—Argumentation, artificial intelligence, assurance cases, evidence-based software engineering, software development lifecycle, verification and validation.**

I. Introduction

AN engineering team needs grounds for accepting a change. A feature request describes a desired outcome. A patch supplies a candidate implementation. Tests and measurements supply observations. The team must explain why those observations support the intended outcome under the conditions in which it will use the system.

AI assistance makes that explanation especially useful when an assistant proposes both a solution and its checks. The same mistaken interpretation can appear in the requirement, implementation and test expectations. Agreement between those artefacts can then reflect their shared assumption. Recording that assumption makes it available for examination and identifies evidence that could change the decision.

The effect of AI assistance on delivery depends on the task and setting. A controlled study by Peng et al. reported faster completion of a bounded programming task with an AI coding assistant [1]. Becker et al. reported longer completion times with early-2025 AI tools among experienced developers working in familiar repositories [2]. These studies concern different populations and tasks. Their results support context-specific evaluation of engineering methods and tools.

This article proposes *argument-driven engineering* as an organising principle. Engineers maintain a versioned case that states the intended outcome, the claims needed for the next decision and the support each claim requires. Unresolved obligations determine the next useful action. That action might produce code, inspect a dependency or distinguish competing fault explanations. The resulting observations can support a decision, expose a defect or leave the claim unresolved.

The proposed replacement concerns how a team directs and accepts work. Requirements analysis, design, implementation, verification, operation and retirement remain necessary activities. Their order follows the dependencies of the decision being considered. This interpretation also accommodates lifecycle models that already iterate and use operational feedback.

The article makes four proposed contributions. It defines a versioned engineering case as the unit connecting work to decisions. It describes a workflow in which unresolved support obligations select engineering actions. It separates candidate generation from substantive assessment and execution authority. It provides an evaluation design for testing whether this approach improves decisions at an acceptable total cost. The worked feature is hypothetical. Repository inspection identifies existing implementation foundations, while the comparative benefits remain unmeasured.

II. Background and Related Practice

A. The software development lifecycle already includes iteration

The software development lifecycle describes how an organisation designs, creates and maintains software. NIST recognises models that include waterfall, spiral and Agile approaches, including combinations with DevOps practices [3]. A proposed successor should therefore address this range of practice.

Royce's 1970 paper presented a sequence of development activities and warned that a simple implementation of the sequence was risky [4]. His discussion included feedback and additional work to reduce late discovery of problems. Treating that paper as an endorsement of an inflexible sequence would obscure its qualification. Boehm's spiral model later made risk examination central to successive development cycles [5].

The Agile Manifesto emphasised working software and responding to change [6]. Continuous delivery connected changes to repeatable build, test and deployment processes [7]. These approaches already provide ways to shorten feedback and revise work. Argument-driven engineering proposes a further organising relation: a team selects an activity because it addresses an identified claim or decision uncertainty. Whether that relation improves an existing process requires evaluation.

B. Evidence-based engineering informs judgement

Kitchenham et al. proposed evidence-based software engineering as a way to combine research evidence with practical

Affiliation and contact details to be added before submission.

TABLE I
TOULMIN ROLES IN AN ENGINEERING CASE

Role	Question answered
Claim	What conclusion should the decision-maker accept?
Grounds	Which obtained observations or established premises support it?
Warrant	Why do those grounds support this conclusion?
Backing	What supports the warrant's reliability and applicability?
Qualifier	Under which conditions and with what strength does the conclusion follow?
Rebuttal	Which exceptions or counterconditions could defeat the inference?

experience and human values in engineering decisions [8]. Research can inform a design choice, while local observations establish properties of the identified system. An engineer must judge how well either source applies to the present task.

This article uses *evidence-based engineering* for that broader commitment to justified judgement. It uses *argument-driven engineering* for the proposed process that records how particular evidence supports a claim and turns gaps in that support into work. An experiment supplies observations. An argument explains the inference from those observations to the conclusion. Keeping that relation explicit allows another engineer to examine its assumptions.

C. Toulmin and assurance cases supply the reasoning structure

Toulmin distinguishes a claim from its grounds and from the warrant connecting them [9]. Backing supports the warrant. A qualifier limits the force or scope of the conclusion. Rebuttal conditions identify circumstances in which the inference would fail. Table I gives the engineering interpretation used here.

Assurance cases already organise claims, arguments and evidence about system properties. OMG's Structured Assurance Case Metamodel provides an established representation for such cases [10]. The contribution proposed here is to use an executable case to direct ordinary engineering work and reassess decisions as the system changes. The present article leaves comparative effectiveness and any broader novelty claim to further study.

III. THE PROPOSED ENGINEERING CASE

A. The decision fixes the required scope

A case begins with a decision and an intended outcome. Examples include accepting a design, authorising a limited deployment or deciding which explanation of a fault to investigate. The case states the system boundary and the consequences that matter. Its claims identify the revision and configuration to which the proposed conclusion applies.

The case then connects each material claim to its supporting claims or evidence requirements. A material claim is one whose failure could change the decision under the agreed criteria. This criterion controls decomposition. A routine documentation repair may need a short rationale and one observed execution. A recovery mechanism with consequential failure modes may need several arguments about composition and operating conditions.

An evidence requirement remains a work obligation until an acquisition produces an interpretable observation. The requirement identifies the source, method and relevant conditions. It states which result would support the claim and which result would weaken it. Recording adverse and inconclusive outcomes prevents a collection plan from becoming an implicit acceptance promise.

B. Five functions retain separate meanings

Candidate generation proposes a requirement, design, implementation or explanation. An AI assistant can perform this function within a bounded task. The proposal's fluency supplies no measurement of its correctness.

Structural checking establishes whether a representation satisfies its declared syntax and contracts. A well-formed argument can still contain an unsupported premise. Structural checking gives later tools an admissible input and makes certain inconsistencies detectable.

Observation records the outcome of an identified acquisition. A successful test execution might report a failed behavioural condition. An acquisition failure means that the requested observation is unavailable. These outcomes lead to different next actions.

Substantive assessment examines the observation's relevance, reliability and inferential strength. For a benchmark, this includes the relationship between the measured workload and the workload named in the claim. For a formal result, it includes the relationship between the model and implementation. The argument records these relationships as claims and assumptions that can themselves require support.

Authority determines which actions a person or system may take. An assessment can support a recommendation while deployment authority remains with an identified owner or policy. The case records the action under consideration so that support for one decision does not silently authorise a broader one.

C. Claims and observations retain their histories

A claim revision records its wording and the reason for the change. Narrowing a claim can be justified when observations establish a smaller scope. The case preserves the original requirement and identifies whether the revised claim still satisfies it. Reviewers can then detect an unexplained reduction in the promised behaviour.

An observation retains its acquisition identity and original time. Its age differs from the interval over which a proposition is asserted. A measurement at one instant supports an interval claim only through an additional argument about persistence or representative sampling. A later system change can weaken that argument while leaving the historical observation intact.

Evidence reuse therefore requires more than a recent timestamp. The source and acquisition request must correspond

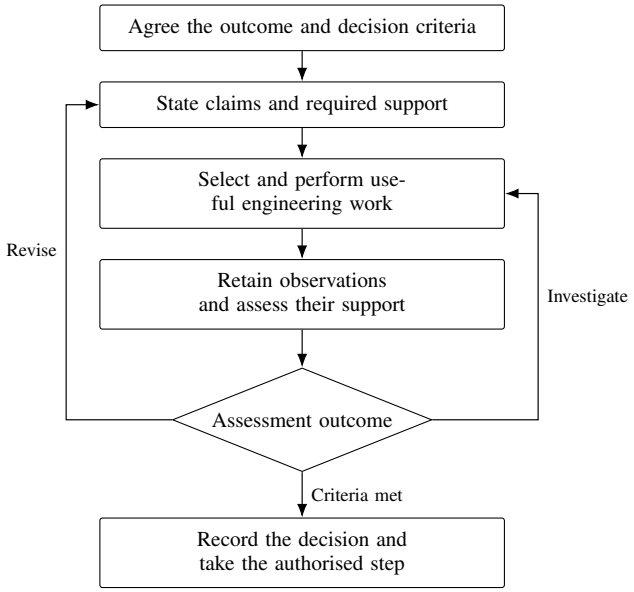


Fig. 1. Proposed workflow. A blocked investigation can end with an explicit unresolved assessment. Relevant changes after a decision reopen the affected claims.

to the present claim. Relevant configuration and execution identities must remain compatible. The declared freshness and applicability conditions must also hold. Shared evidence can support several claims, but those uses retain their dependence on the same source.

IV. The Engineering Workflow

Figure 1 shows the proposed loop. Engineers agree the decision criteria before judging the candidate. They identify material claims and examine existing support. They then choose work that could change the decision or make a necessary dependent action possible.

The next action depends on the gap. A missing model premise can require inspection. Competing diagnoses can require a discriminating experiment. A known implementation defect can require a patch. Repeating a test adds value only when the repetition addresses uncertainty relevant to the decision.

Every delegated action states its permitted effects and resource limits. Its result identifies the implementation or instrument used and the outputs retained. An assistant that reaches its limit returns the remaining gap. The workflow can then assign another bounded action or record that the investigation remains blocked.

Assessment has three broad outcomes. The evidence can satisfy the criteria for the scoped decision. A counterexample or failed assumption can require revision. Missing or inconclusive support can leave the decision unresolved. These categories describe different states of knowledge and should remain distinguishable in reports and automation.

Decision criteria can differ across successive deployment scopes. Evidence from an isolated test environment can support a controlled trial when the corresponding risks and recovery arrangements are acceptable. Observations from that trial can then support a wider rollout. This sequence lets evidence from operation inform later decisions without making a production observation a prerequisite for every initial deployment.

After deployment, the case records reassessment conditions. A change in an implementation dependency can reopen a compatibility claim. A workload change can reopen a performance claim. A newly observed failure can challenge a warrant about recovery. The proposed scheduler uses an explicit impact model to locate affected claims and obtain the support required for the next decision.

V. Worked Feature: Retry-Safe Job Submission

A. Outcome and bounded claim

The hypothetical feature allows a client to recover a job's identity after a submission response is lost. The authenticated tenant and a client-supplied request key identify a logical submission. Equivalent validated payloads using an active identity return the same durable job identifier. An incompatible payload receives a conflict response.

The identity remains active for 24 hours from the first committed submission. A retry preserves that expiry. After expiry, a request can establish a new submission. Payload equivalence covers every field that changes the job's meaning, including the relevant API interpretation. The implementation must define that comparison before tests can assess it.

The principal implementation claim is that competing API submissions create at most one durable job for the same active tenant and request key. A separate response claim requires successful equivalent submissions to return that job's identifier. Job dispatch and external side effects need their own arguments because the existence of one durable record alone leaves their behaviour unresolved.

B. Warrant and implementation work

The proposed warrant relies on shared uniqueness and atomic persistence. Every relevant creation path uses the same uniqueness rule. A transaction commits the job together with the retained request identity and payload. Competing requests resolve against the committed identity. The datastore's transaction and durability guarantees supply backing only for the configuration in which the service uses them.

This warrant produces implementation obligations. The engineer must inspect every creation path and establish how transactions handle a concurrent conflict. The code must compare an existing identity's payload before returning its job identifier. Expiry and reuse must remain atomic with competing requests. A failover configuration that can lose acknowledged state requires an explicit qualification or a revised design.

Table II states proposed checks. Each row names an observation that could expose a defect. The acceptance decision also requires inspection of path coverage and the persistence assumptions. A successful fixture suite alone leaves those relationships to be assessed.

TABLE II
PROPOSED EVIDENCE FOR RETRY-SAFE SUBMISSION

Condition exercised	Required observation
Concurrent equivalent requests	One durable job exists. Successful responses identify that job.
Response lost after commit	A retry recovers the committed job identifier.
Process stops before commit	A later retry permits one eventual committed creation.
Incompatible payloads compete	One payload establishes the identity. Incompatible requests receive conflicts.
Same key across tenants	Each tenant obtains an independent identity.
Requests race with expiry	Creation and reuse follow the specified time boundary.

C. Decision and later reassessment

The feature case can justify a limited deployment after the required implementation and integration claims receive support. The case states the remaining operational uncertainty and identifies how the trial will examine it. A failed test directs a repair. An unavailable failover observation leaves the corresponding claim unresolved.

A later datastore change reopens the durability assumption and dependent conclusions. The original test results continue to describe the tested revision. An impact assessment must establish which observations still apply and which require replacement. The proposed method makes that reassessment a normal engineering action with a recorded cause.

VI. Implementation Foundations and Required Extensions

A. EngArg constructs candidate arguments and acquisition plans

EngArg develops extended engineering arguments from claims and optional evidence. Its EAL translation preserves scoped claims and review obligations. Conversion provides an executable projection whose source and context can be retained. Natural-language premises still need evidence and interpretation [11].

The optional tooling stage can generate missing collectors against frozen contracts and fixtures. Existing collectors can supply configured acquisition paths. Passing the frozen tests establishes the checked tool behaviour. Applicability to the intended measurement and sufficiency for the engineering inference remain separate obligations. This separation matters when AI produces both the collector and its test contract.

B. EAL-MCP collects and assesses declared support

EAL-MCP provides typed reasoning contracts and a durable evidence runtime. Registered assessment plans the declared support, objection and defence dependencies. It selects compatible stored observations and acquires missing or expired readings. Reused observations retain their original measurement age. Source revisions and method identities identify the assessment's declared basis [12].

The inspected runtime distinguishes failed acquisition from measured negative values. Its application contracts also distinguish a completed assessment from a supported claim. Support remains conditional on declared scope, observations, assumptions and inference rationales. These boundaries permit an assistant to report an unresolved obligation without treating it as a refutation of the proposition.

C. A general development process requires further integration

The pinned repositories provide foundations for argument construction and assessment. A general engineering process also needs an orchestrator that assigns arbitrary feature, repair and investigation work. It needs a mapping from system changes to affected claims. Event integration must schedule reassessment when the relevant conditions change.

The process must connect product objectives to technical criteria and make execution authority explicit. An acceptable implementation can still address an unnecessary requirement. A supported claim can still concern an action outside an assistant's delegated authority. Stakeholders must retain ownership of those distinct judgements.

This account concerns repository contracts and selected source inspection at the cited revisions. It establishes the architecture available for a pilot. It supplies no comparative evidence that the combined process improves delivery, reduces defects or generalises across engineering disciplines.

VII. Evaluation Template

A. Questions and comparison

The primary question is whether argument-driven engineering improves the quality of acceptance decisions at an acceptable total cost. A second question concerns reassessment: does the process identify and reopen conclusions whose support a later change weakens? These questions determine the measurements and the comparison.

The proposed pilot compares the team's existing workflow with the same workflow augmented by executable engineering cases. Both conditions use the same permitted AI capabilities and development tools. Tasks include feature work and defect repair, with an operational change where the setting permits. A feasibility pilot first checks whether the protocol and measures can be applied consistently. Its data then inform the size of a subsequent comparative study.

B. Assignment and independent assessment

Assignment should account for developer experience and task difficulty. Randomisation within suitable developer and task blocks can limit systematic differences. Repeating an identical task with the same developer introduces learning, so the protocol should use independent or carefully matched tasks and state how it handles carry-over. Repository differences require either blocking or an explicit limitation on generalisation.

An assessor reviews the submitted outcome against criteria fixed independently of the assistant's final argument. Where feasible, the assessor remains unaware of the assigned process. The review checks requirement coverage and the validity of

TABLE III
RESULTS TEMPLATE: ALL OUTCOMES REMAIN UNMEASURED

Measure	Existing workflow	Proposed workflow
Unsupported acceptance rate	To be measured	To be measured
Defects found after acceptance	To be measured	To be measured
Engineering and review effort	To be measured	To be measured
Elapsed task time	To be measured	To be measured
Execution cost	To be measured	To be measured
Affected claims correctly reopened	To be measured	To be measured

the reported evidence. Known-answer tasks and independently seeded defects can test whether the process rejects an attractive but defective candidate. Live tasks require a stated follow-up period for failures discovered after acceptance.

The study retains original requirements and all authorised revisions. It records unresolved outcomes as such and reports task abandonment. This prevents a method from appearing effective because it quietly weakens the requirement or excludes its difficult cases.

C. Measures and analysis

Table III is the results template. The final protocol must define each denominator and observation window before data collection. Total cost includes engineering effort and review effort, along with model and tool execution charges. Elapsed time includes blocked intervals when they form part of the delivery objective.

The analysis should report effect estimates with uncertainty and account for repeated observations from the same developer or repository. The protocol must specify which quality differences count as practically important and which cost increase the team would accept. Small pilot samples primarily inform feasibility and uncertainty. They offer a limited basis for general adoption claims.

Raw counts of closed claims and collected observations measure activity. The outcome of interest is whether the process supports a better decision. A large case can contain redundant support, while one discriminating observation can resolve the uncertainty that matters. The study should examine this relationship through independent case review.

VIII. Limitations and Adoption

An argument can rationalise a poor requirement. Its completeness relative to declared dependencies leaves undeclared requirements open. Independent assessment of intended use and requirement coverage therefore remains necessary. A method that makes technical reasoning explicit still depends on sound choices about whose outcomes matter.

Evidence can also share errors. Several reports generated from the same measurement provide several uses of one source. A generated test can repeat the implementation's misunderstanding. The case should expose these dependencies and use independent checks where their expected error-detection value justifies their cost.

Maintaining detailed cases can consume more effort than a decision warrants. Teams should begin with the material uncertainty and expand the case when the consequences or disagreements require it. A short claim with linked evidence can be sufficient for a bounded change. The method's evaluation must include the cost of maintaining its own records.

An incremental adoption path uses existing issues and pull requests to hold the engineering case. CI provides acquired observations. Review records the warranted conclusion and the permitted next step. Operational events can later trigger reassessment. This route permits a comparison with existing practice before the organisation changes its wider delivery process.

IX. Conclusion

Argument-driven engineering proposes that a versioned case should direct work towards justified engineering decisions. Evidence requirements become implementation or investigation obligations. Obtained observations support scoped conclusions through explicit reasoning. AI assistants can perform bounded parts of that work while engineers and authorised policies retain responsibility for consequential decisions.

EngArg and EAL-MCP provide an existing basis for testing this proposal. The next development task is to connect their argument and assessment capabilities to general engineering actions and relevant system changes. The next research task is to measure decision quality and total cost against an identified existing workflow. Those results will determine whether the proposed organising principle warrants wider use.

Template Use and Research Status

This document is a populated IEEE-style article template for a perspective and method proposal. The worked feature and evaluation protocol are prospective. Table III contains no empirical results. Before submission, the author should select an IEEE venue, replace the affiliation note and apply that venue's current instructions [13]. Any completed study should replace the prospective protocol and retain its deviations and limitations.

References

[1] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, "The impact of AI on developer productivity: Evidence from GitHub Copilot," arXiv:2302.06590v1, Feb. 2023. [Online]. Available: https://arxiv.org/abs/2302.06590v1

[2] J. Becker, N. Rush, B. Barnes, and D. Rein, "Measuring the impact of early-2025 AI on experienced open-source developer productivity," arXiv:2507.09089v2, Jul. 2025. [Online]. Available: https://arxiv.org/abs/2507.09089v2

[3] M. Souppaya, K. Scarfone, and D. Dodson, "Secure software development framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities," National Institute of Standards and Technology, NIST Special Publication 800-218, Feb. 2022. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-218

[4] W. W. Royce, "Managing the development of large software systems," in *Proceedings of IEEE WESCON*, Aug. 1970, pp. 1–9. [Online]. Available: https://www-scf.usc.edu/~csci201/lectures/Lecture11/royce1970.pdf

[5] B. W. Boehm, "A spiral model of software development and enhancement," *Computer*, vol. 21, no. 5, pp. 61–72, May 1988. [Online]. Available: https://doi.org/10.1109/2.59

[6] K. Beck, M. Beedle, A. van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, J. Kern, B. Marick, R. C. Martin, S. Mellor, K. Schwaber, J. Sutherland, and D. Thomas, "Manifesto for agile software development," 2001, accessed: Oct. 26, 2026. [Online]. Available: https://agilemanifesto.org/

[7] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010. [Online]. Available: https://www.informit.com/store/continuous-delivery-reliable-software-releases-through-9780321601919

[8] B. A. Kitchenham, T. Dybå, and M. Jørgensen, "Evidence-based software engineering," in *Proceedings of the 26th International Conference on Software Engineering*. IEEE Computer Society, 2004, pp. 273–281. [Online]. Available: https://doi.org/10.1109/ICSE.2004.1317449

[9] S. E. Toulmin, *The Uses of Argument*. Cambridge, U.K.: Cambridge University Press, 2003, updated edition.

[10] Object Management Group, *Structured Assurance Case Metamodel*, Object Management Group, Oct. 2023, version 2.3. [Online]. Available: https://www.omg.org/spec/SACM/2.3/PDF

[11] E. Miller, "EngArg: Engineering argument construction and evidence planning," GitHub repository, revision 09f6b77617f0a93506fb4fc55d2ebdb5b9c39902, 2026, source inspection, Oct. 26, 2026. [Online]. Available: https://github.com/emmett08/engarg/tree/09f6b77617f0a93506fb4fc55d2ebdb5b9c39902

[12] ——, "EAL/3: Engineering argument assessment and evidence runtime," GitHub repository, revision bce2c701c2eef03fb23004714045d8601555c36b, 2026, source inspection, Oct. 26, 2026. [Online]. Available: https://github.com/emmett08/eal-mcp/tree/bce2c701c2eef03fb23004714045d8601555c36b

[13] IEEE, "IEEE article templates," IEEE Author Center Journals, accessed: Oct. 26, 2026. [Online]. Available: https://journals.ieeeauthorcenter.ieee.org/create-your-ieee-journal-article/authoring-tools-and-templates/tools-for-ieee-authors/ieee-article-templates/